

Hoja 1 — Guía rápida para pensar recursivamente

Objetivo: pasar del enunciado a una función recursiva correcta.

- 1) Enunciado (problema en tus palabras): _____
- 2) Estructura de datos principal: [lista/cadena/matriz/árbol/grrafo/otra]
- 3) Estado mínimo (parámetros que describen 'dónde voy'):
- 4) Caso base (cuándo respondo directo):
- 5) Progreso hacia el caso base (qué se reduce en cada llamada):
- 6) Llamadas recursivas (hijos del problema):
- 7) Cómo combinar la respuesta de los hijos:
- 8) Pruebas rápidas (3 casos, incluyendo borde):

Hoja 2 — Checklist de seguridad (antes de ejecutar)

- ☐ El caso base se alcanza con entradas pequeñas (vacío, 1 elemento).
- ☐ En cada llamada hay progreso real hacia el caso base.
- ☐ No modifíco la entrada original sin necesidad (usar copias si aplica).
- ☐ Escribí 3 pruebas: caso típico, borde mínimo y borde extraño.
- ☐ Puedo trazar parámetros para ver el orden de llamadas (prints).

Hoja 3 — Plantilla patrón ÍNDICE (listas/cadenas)

Idea: recorrer por posición aumentando un puntero *i* (o dos punteros *l*, *r*).

Esqueleto 1 (índice hacia adelante):

```
def solve(lista, i=0):  
    # 1) caso base  
    if i == len(lista):  
        return ... # valor trivial (0, True, "", etc.)  
    # 2) paso recursivo: usar lista[i] y avanzar  
    respuesta_hijo = solve(lista, i + 1)  
    # 3) combinar  
    return ... # combina lista[i] con respuesta_hijo
```

Esqueleto 2 (dos punteros extremos):

```
def solve2(s, l=0, r=None):  
    if r is None: r = len(s) - 1  
    if l > r: # o l >= r según el problema  
        return ...  
    # usar s[l] y s[r]  
    return ... # combina solve2(s, l+1, r-1) con lo necesario
```

Hoja 4 — Plantilla DIVIDE & CONQUER (partir y combinar)

Parte el problema en subproblemas independientes, resuelve y combina.

```
def dyc(a):
    # 1) caso base: tamaño pequeño
    if len(a) <= 1:
        return a # o respuesta trivial
    # 2) dividir
    mid = len(a) // 2
    izq = a[:mid]; der = a[mid:]
    # 3) conquistar (resolver subproblemas)
    R_izq = dyc(izq)
    R_der = dyc(der)
    # 4) combinar
    return combinar(R_izq, R_der)

def combinar(L, R):
    ... # define cómo unir las respuestas (merge, max, etc.)
```

Hoja 5 — Plantilla INCLUIR / EXCLUIR (decisión binaria)

En cada posición decides no tomar o tomar el elemento.

```
def incluir_excluir(nums, i=0, actual=None, res=None):
    if res is None: res = []
    if actual is None: actual = []
    # 1) caso base
    if i == len(nums):
        res.append(actual.copy()); return res
    # 2) decisión A: NO incluir nums[i]
    incluir_excluir(nums, i + 1, actual, res)
    # 3) decisión B: SÍ incluir nums[i]
    actual.append(nums[i])
    incluir_excluir(nums, i + 1, actual, res)
    actual.pop()
    return res
```

Hoja 6 — Traza manual (árbol de llamadas)

Cómo hacerlo:

- 1) Escribe el estado que cambia (i, l..r, actual).
- 2) Dibuja las llamadas como nodos de arriba hacia abajo.
- 3) Marca el caso base con un ✓ y anota qué retorna.
- 4) Indica cómo cada nodo combina las respuestas de sus hijos.

Espacio de trabajo:

Hoja 7 — Prueba de escritorio (paso a paso)

Entrada: _____

Parámetros iniciales: _____

Llamada 1 → parámetros: _____ → retorno: _____

Llamada 2 → parámetros: _____ → retorno: _____

Llamada 3 → parámetros: _____ → retorno: _____

Resultado final esperado: _____

Hoja 8 — Errores frecuentes y cómo evitarlos

- Olvidar el caso base o tenerlo inalcanzable.
- No avanzar hacia el caso base (parámetro que no cambia).
- Modificar listas en sitio sin control (usar copias o deshacer append).
- Mezclar retornos y efectos (decide si devuelves o imprimes/guardas).
- No probar con casos vacío y tamaño 1.

Hoja 9 — Mini-retos guiados (solo patrones)

Reto A (ÍNDICE): cuenta cuántas veces aparece x en la lista.

Reto B (DyC): divide la lista en dos y combina resultados de forma definida.

Reto C (Incluir/Excluir): genera todas las sublistas de tamaño k .

Añade casos de prueba y una traza breve.

Hoja 10 — Explica tu solución (metacognición)

- 1) ¿Cuál es tu caso base y por qué es correcto?
- 2) ¿Qué parámetro garantiza el avance hacia ese caso base?
- 3) ¿Cómo combinas las respuestas de las subllamadas?
- 4) Traza 3–4 llamadas con retornos.
- 5) Escribe dos pruebas y di por qué cubren casos borde.